SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lab Practical 11

DevOps and Cloud Hosting Platforms

Lab Practical Number	Lab 11
Lab Title	DevOps and Cloud Hosting Platforms
Related Lecture	Lecture 11 – DevOps and Cloud Hosting Platforms
Duration	2 Hours (in-lab)
Learning Outcomes	LO3 – Best practices for CI/CD, quality & deployment • LO4 – Select & justify approaches
Platforms	GitHub • Vercel / Netlify (frontend) • Render / Railway (backend) • Neon / Supabase (database)
Mode	Individual work • Word document + deployment URLs submitted to CourseWeb (no secret values)

Introduction

This lab is where your application leaves your laptop and goes live. In the lecture you studied DevOps culture and the build → test → deploy → monitor workflow, cloud hosting platforms, deploying frontends and backend APIs, hosting a database, production configuration with environment variables, and monitoring deployed applications. In this 2-hour session you will deploy a full-stack application to the cloud, connect its tiers, and review its logs and status.

Everything you need comes from Lecture 11. Base your deployment steps, platform choices, environment configuration, monitoring explanation, and DevOps reflection on the lecture content and the class examples (the delivery pipeline, “it works on my machine” dying with consistent environments, and “never put secrets in code”).

> **CONTINUITY WITH LABS 02–04 & 08**
>
> You already built the pieces: the CampusEats React frontend (Lab 02), the ASP.NET Core API (Lab 03), the PostgreSQL database and auth (Lab 04), and a GitHub repository with CI (Lab 08). This lab is the final step — deploying that work so real users can reach it.

> **WHAT YOU MUST COMPLETE (REALISTIC FOR 2 HOURS)**
>
> Deploying a frontend is fast and reliable — **everyone should complete Task 02**. The backend and database (Tasks 03–04) are guided; complete them if your application and time allow, otherwise **document the steps and explain the flow conceptually** for marks. A working frontend plus a clear explanation of the rest is a full submission.

Lab Objectives

By the end of this lab, you should be able to:

- **Prepare an application for deployment** creating a production build and identifying configuration that must become environment variables (LO3).
- **Deploy a frontend to a cloud platform** such as Vercel or Netlify, from a GitHub repository (LO3).
- **Deploy a backend API to a cloud platform** such as Render or Railway, with the correct port, CORS, and environment settings (LO3).
- **Host a database and configure connections** using a managed PostgreSQL service and environment variables (LO3).
- **Test, monitor, and read logs** for a deployed application, confirming its status and behaviour (LO3).
- **Justify platform choices and reflect on DevOps** for real-world deployment requirements (LO4).

Required Tools / Accounts

- **A GitHub account —** with your application in a repository (from earlier labs).
- **A frontend host account —** Vercel (vercel.com) or Netlify (netlify.com) — free tier.
- **A backend host account —** Render (render.com) or Railway (railway.app) — free tier.
- **A managed database account —** Neon (neon.tech) or Supabase (supabase.com) — free PostgreSQL.
- **Node.js and/or the .NET 8 SDK —** installed locally to create production builds.
- **A browser and a terminal —** for the platform dashboards and Git/CLI commands.

> **NO PRIOR PROJECT? USE A STARTER**
>
> If you do not have a completed application, create a minimal frontend to deploy: `npm create vite@latest campuseats-web -- --template react`, then `npm install` and push it to GitHub. You can still complete Task 02 fully and explain Tasks 03–04.

Lab Time Plan (120 minutes)

Time	Activity	Output
0:00 – 0:08	Briefing & recap of Lecture 11 (DevOps, cloud, monitoring)	Ready to deploy
0:08 – 0:23	Task 01 — App & repository preparation (production build)	Build + env vars ready
0:23 – 0:45	Task 02 — Frontend deployment (Vercel / Netlify)	Live frontend URL
0:45 – 1:08	Task 03 — Backend API deployment (Render / Railway)	Live API URL (or steps)
1:08 – 1:26	Task 04 — Database hosting & environment configuration	Cloud DB + env vars
1:26 – 1:44	Task 05 — Testing, monitoring & logging	Tested + logs reviewed
1:44 – 1:54	Task 06 — DevOps reflection & platform justification	Reflection + table
1:54 – 2:00	Quick test + screenshots and submit	Submitted .docx

> **HOW TO USE THIS PLAN**
>
> These times are a guide. Complete the frontend deployment first (Task 02) so you have a live URL, then move to the backend and database. Capture screenshots of every dashboard and status page as you go.

Mini Project Scenario

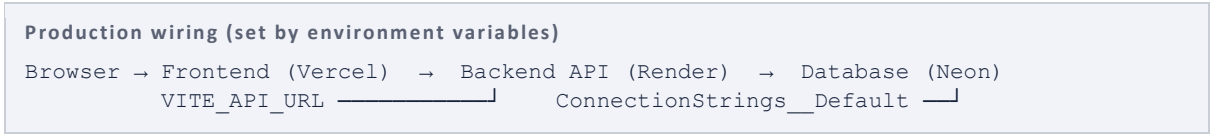
DEPLOY THE "CAMPUSEATS" FULL-STACK APPLICATION

CampusEats is ready for users. Your job is the **deployment**: put the React frontend on a static host, the ASP.NET Core API on a backend host, and the database on a managed PostgreSQL service — then wire them together with environment variables so the live frontend talks to the live API, which talks to the live database.

The three tiers and where each one goes

Tier	What it is	Suitable host (pick one)
Frontend	React / Vite single-page app	Vercel or Netlify
Backend API	ASP.NET Core REST API	Render or Railway
Database	PostgreSQL	Neon, Supabase, or Railway PostgreSQL

How the deployed tiers connect



TASK 01 • 15 minutes • LO3

Application & Repository Preparation

Before deploying, get the application into a deployable shape: a working production build, secrets removed from code, and configuration moved to environment variables.

Step-by-step instructions

1. Make sure your application is pushed to a **GitHub** repository (cloud hosts deploy directly from GitHub).
2. Confirm the frontend produces a production build locally. For a Vite app:

Terminal (frontend)

```
$ npm install
$ npm run build       # outputs the production site to dist/
$ npm run preview     # optional: serve dist/ locally to check
```

3. Replace any hard-coded backend URL in the frontend with an **environment variable** so it can differ between local and production:

src (frontend)

```
// frontend: read the API base URL from the environment
const API = import.meta.env.VITE_API_URL;
// use it for requests:
fetch(`${API}/api/menu`)
```

4. Check your `.gitignore` excludes `node_modules`, `.env`, `bin`, and `obj` — these must never be committed.
5. List the environment variables each tier will need (you will set their real values on the platforms): `VITE_API_URL` (frontend); `ConnectionStrings__Default`, `Jwt__Key` (backend).

EXPECTED OUTPUT

The app on GitHub, a successful local `npm run build`, the API URL read from `VITE_API_URL`, and a short list of the environment variables each tier needs.

TASK 02 • 22 minutes • LO3 • COMPLETE THIS ONE

Frontend Deployment (Vercel / Netlify)

Deploy the React frontend to a static host. These platforms build directly from your GitHub repository and give you a live HTTPS URL on a global CDN — the standard way to host a single-page app.

Step-by-step instructions (Vercel)

1. Sign in to `vercel.com` with GitHub, then `Add New → Project` and import your repository.
2. Confirm the build settings (Vercel auto-detects Vite):

Setting	Value
Framework Preset	Vite
Install Command	npm install
Build Command	npm run build
Output Directory	dist

3. Add an environment variable `VITE_API_URL`. For now set it to your backend URL if you have one, or a placeholder you will update after Task 03.
4. Click `Deploy`. When the build finishes you receive a live URL such as `https://campuseats-web.vercel.app`.
5. Open the live URL and confirm the app loads.

NETLIFY ALTERNATIVE

On `netlify.com`: `Add new site → Import an existing project`, pick the repo, set *Build command* = `npm run build` and *Publish directory* = `dist`. Add the same `VITE_API_URL` variable under Site settings → Environment.

EXPECTED OUTPUT

A live, public frontend URL. Screenshots of the deployment dashboard, the successful build/deploy status, the environment-variable configuration (with the value hidden), and the running site in the browser.

TASK 03 • 23 minutes • LO3 • COMPLETE OR DOCUMENT

Backend API Deployment (Render / Railway)

Deploy the ASP.NET Core API to a backend host. A backend needs three things to run in the cloud: it must listen on the platform's **port**, allow your frontend's origin via **CORS**, and read its secrets from **environment variables**.

Step-by-step instructions (Render, using Docker)

1. Add a `Dockerfile` to the API project so Render can build it:

Dockerfile (API project root)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENV ASPNETCORE_URLS=http://0.0.0.0:8080   # listen on platform port
EXPOSE 8080
ENTRYPOINT ["dotnet", "CampusEats.Api.dll"]
```

2. Allow the deployed frontend to call the API. In `Program.cs`, add a CORS policy and use it:

src: Program.cs

```
builder.Services.AddCors(o => o.AddPolicy("Frontend", p =>
    p.WithOrigins("https://campuseats-web.vercel.app")
     .AllowAnyHeader().AllowAnyMethod()));
// ... after the app is built:
app.UseCors("Frontend");   // before MapControllers()
```

3. Commit and push. On `render.com`: `New → Web Service`, connect the repo, choose the **Docker** runtime, and create the service.
4. Add the backend environment variables in Render's `Environment` tab (see Task 04 for the values), then deploy.
5. When the deploy succeeds, open the service URL and test a public endpoint, e.g. `https://campuseats-api.onrender.com/api/menu` or `/swagger`.

RAILWAY ALTERNATIVE

On `railway.app`, `New Project → Deploy from GitHub repo` auto-detects .NET (Nixpacks) — often no Dockerfile needed. Set the same environment variables under the service's `Variables` tab.

EXPECTED OUTPUT

A deployed backend URL with a working endpoint (or, if not completed, the Dockerfile, the CORS code, and a written description of the deploy steps). Screenshots of the deploy dashboard and a successful API response.

TASK 04 • 18 minutes • LO3 • COMPLETE OR DOCUMENT

Database Hosting & Environment Configuration

Host the database on a managed PostgreSQL service and point the deployed backend at it — entirely through environment variables, so no connection string or secret ever sits in your code.

Step-by-step instructions

1. Create a free PostgreSQL database on `neon.tech` (or Supabase / Railway). Copy the connection details it gives you.
2. Set the backend environment variables on your host (Render/Railway). Managed Postgres requires SSL, so include the SSL settings:

Environment variable	Example value (use your own; keep secret)
`ConnectionStrings__Default`	`Host=<neon-host>;Database=campuseats;Username=<user>;Password=<pwd>;SSL Mode=Require;Trust Server Certificate=true`
`Jwt__Key`	a long random production secret (different from dev)
`Jwt__Issuer`	`CampusEats`
`Jwt__Audience`	`CampusEatsUsers`

Note: the double underscore `__` maps to nested configuration in .NET, so `ConnectionStrings__Default` overrides the `ConnectionStrings:Default` value from `appsettings.json`.

3. Apply your EF Core migrations to the cloud database (run locally, pointed at the cloud connection string), so the tables exist:

Terminal (API project)

```
# set the connection string for this command, then:
$ dotnet ef database update
```

4. Finally, set the frontend's `VITE_API_URL` (from Task 02) to the deployed backend URL and redeploy the frontend so the live tiers are connected.

EXPECTED OUTPUT

A managed PostgreSQL database; the backend configured to use it via environment variables; migrations applied; and the frontend pointing to the live backend. Screenshots of the database dashboard and the environment-variable configuration (values hidden).

PRODUCTION CONFIG RULE (FROM THE LECTURE)

Configuration that changes between environments — and every secret — belongs in **environment variables**, not in code or `appsettings.json`. Use different values for dev, test, and production. Never commit a `.env` file or a real connection string.

TASK 05 • 18 minutes • LO3

Testing, Monitoring & Logging

A deployment is not done until you have confirmed it works and can observe it. This is the `Monitor` stage of the DevOps pipeline.

Step-by-step instructions

1. Test end-to-end: open the live frontend URL and perform an action that calls the API (for example, view the menu). Confirm real data appears.
2. Test the API directly: open the deployed API endpoint (e.g. `/api/menu` or `/swagger`) in the browser or Postman and check the response and status code.
3. Review deployment status: on each platform dashboard, confirm the latest deploy shows **success** (green). Note the build time and the commit it deployed.
4. Review logs: open the `Logs` view on Vercel (build/function logs) and Render/Railway (runtime logs). Find one meaningful log line (e.g. a request, a startup message, or an error).
5. If something fails, use the logs to diagnose it — see the troubleshooting box below.

COMMON DEPLOYMENT PROBLEMS & FIXES

`CORS error in the browser` → add the exact frontend origin to the backend CORS allow-list and redeploy.

`Frontend loads but no data` → VITE_API_URL is wrong or unset; fix it and redeploy the frontend.

`First request very slow` → free-tier cold start; the service sleeps when idle and wakes on the first request.

`DB connection fails` → missing SSL Mode=Require, wrong connection string, or migrations not applied.

`Mixed-content blocked` → an HTTPS frontend cannot call an HTTP backend; the backend must be HTTPS.

EXPECTED OUTPUT

Evidence that the deployed application works end-to-end, the API responds, and the deploy status is green. Screenshots of the working app, an API response, and a logs/monitoring page.

TASK 06 • 10 minutes • LO3, LO4

DevOps Reflection & Platform Justification

Step back and connect what you did to the DevOps ideas from the lecture, and justify your platform choices.

Part A — Justify your platform choices (LO4)

Complete the table: name the platform you used for each tier and give one reason it suits that tier.

Tier	Platform you used	Why it suits this tier (one reason)
Frontend		
Backend API		
Database		

Part B — DevOps reflection

Write 4–5 sentences covering: the workflow you followed (code → GitHub → build → deploy → monitor); how environment variables kept secrets out of your code and let the same build run in different environments; what monitoring/logs told you; and one DevOps benefit you experienced (e.g. automated builds, or consistent environments removing “it works on my machine”).

CONNECT IT TO CI/CD (LAB 08)

Lab 08 added Continuous Integration (automated checks on every push). Cloud hosts add Continuous Deployment: when you push to the main branch, the platform automatically rebuilds and redeploys. Together they are the CI/CD pipeline — commit to production with no manual file copying.

EXPECTED OUTPUT

A completed platform-justification table and a short, specific DevOps reflection grounded in what you actually did in this lab.

QUICK TEST • 6 minutes

Knowledge Check

Answer all questions in your Word document. Write the question number and your answer. Q1–Q5 are multiple choice; Q6 is about environment variables; Q7 is about deployment; Q8 is a monitoring scenario.

Multiple choice (choose one)

Q1. DevOps primarily aims to:

(a) replace developers with operations staff
(b) unite development and operations with automation to deliver faster and more reliably
(c) remove all testing
(d) write more documentation

Q2. Which platform type best suits a static React / Vite frontend?

(a) a relational database service
(b) a container registry
(c) a frontend/static host such as Vercel or Netlify
(d) a message queue

Q3. Where should production secrets (DB password, JWT key) be stored?

(a) hardcoded in the source
(b) committed in appsettings.json
(c) in environment variables configured on the hosting platform
(d) in the README

Q4. In a Build → Test → Deploy → Monitor pipeline, "Monitor" means:

(a) compile the code
(b) observe the running app's logs, metrics, and errors
(c) write unit tests
(d) merge a pull request

Q5. Continuous Deployment (CD) means:

(a) automatically releasing builds to an environment after checks pass
(b) manually copying files to a server
(c) deleting old branches
(d) running only on localhost

Environment variables

Q6. Give two reasons to use environment variables for configuration instead of hard-coding values in the source.

Deployment

Q7. For a Vite React app, what is the build command and which folder contains the production files to deploy?

Monitoring scenario

Q8. Your deployed frontend loads, but its API calls fail with CORS errors in the browser console. Name two things you would check to fix this.

SUBMIT YOUR QUICK-TEST ANSWERS INSIDE THE SAME WORD DOCUMENT

> Place your quick-test answers in a clearly labelled “Quick Test” section after your Task 01–06 answers.

Submission Instructions

Prepare a single Microsoft Word document containing all of your work for this lab, and a project ZIP if your lecturer requests it.

> **DO NOT SUBMIT SECRETS**
>
> Your document must **not** contain real passwords, secret keys, database passwords, JWT secrets, or API tokens. When you screenshot environment-variable settings, **hide the values** (most dashboards mask them by default).

Your Word document must include:

- Student name
- IT number
- Lab practical number (Lab 11)
- Your GitHub repository link
- Your deployed frontend URL (if completed)
- Your deployed backend API URL (if completed)
- Screenshots of the cloud platform deployment dashboard
- Screenshots of the successful build / deployment status
- Screenshots of the environment-variable configuration (values hidden)
- Screenshots of the deployed application or API being tested
- Screenshots of the logs or monitoring page
- A brief explanation of the DevOps workflow followed
- A brief explanation of the selected cloud hosting platform
- A brief explanation of production configuration and environment variables
- Your Quick Test answers (Q1–Q8)

File naming and upload:

1. Rename the Word document using your IT number: `ITXXXXXXXX_Lab11.docx`
2. Example: `IT22123456_Lab11.docx`
3. If submitting the project, name the ZIP: `ITXXXXXXXX_Lab11_Project.zip` (exclude `node_modules`, `bin`, `obj`, `.env`).
4. Submit the Word document (and ZIP if required) to **CourseWeb** before the end of the 2-hour lab session.
5. **Late submissions will not be accepted** unless prior approval is given by the lecturer.

> **STUDENT INSTRUCTION**
>
> You must refer to the provided Lecture 11 material when completing the lab tasks. Your deployment steps, cloud-platform selection, environment configuration, monitoring explanation, and DevOps reflection should be based on the lecture content, class explanations, and the practical examples discussed during the lecture.

Evaluation Criteria (Marking Guide — 20 Marks)

For lecturer reference. Marks reward a real deployment, correct production configuration, evidence of monitoring, and justified platform choices. A deployed frontend plus a clear, correct explanation of the backend/database flow can earn full conceptual marks where hands-on time runs out.

Component	Marks	What earns full marks
Task 01 — Preparation	2	App on GitHub; successful production build; config moved to environment variables.
Task 02 — Frontend deployment	4	Live frontend URL on Vercel/Netlify; correct build settings; env var configured.
Task 03 — Backend deployment	4	Deployed API with working endpoint (or correct Dockerfile/CORS + documented steps); port & CORS handled.
Task 04 — Database & env config	3	Managed PostgreSQL; backend uses it via env vars; migrations applied; secrets not in code.
Task 05 — Testing & monitoring	3	End-to-end test passes; deploy status green; logs/monitoring reviewed with a screenshot.
Task 06 — Reflection & justification	2	Platform-justification table complete; specific, accurate DevOps reflection.
Quick Test (Q1–Q8)	2	Q1–Q5 correct (b, c, c, b, a); Q6–Q8 show correct understanding.
Total	**20**	

FOR LECTURER REFERENCE ONLY

Expected Answers & Marking Notes

Quick Test — answer key

Q	Answer	Why
Q1	(b) unite dev & ops	DevOps joins development and operations with automation for faster, reliable delivery.
Q2	(c) static host	A static/frontend host (Vercel/Netlify) serves the built SPA over a CDN.
Q3	(c) env variables	Secrets belong in platform environment variables, never in code or Git.
Q4	(b) observe the app	Monitoring watches logs, metrics, and errors of the running application.
Q5	(a) auto-release	CD automatically releases builds to an environment after checks pass.
Q6	Two valid reasons	e.g. different values per environment (dev/prod) AND keeping secrets out of source/Git.
Q7	npm run build → dist	Vite builds to the dist/ folder, which is what you deploy.
Q8	CORS + API URL	Check the backend CORS allow-list includes the frontend origin, and that VITE_API_URL points to the correct live backend (and the backend is awake / HTTPS).

Task acceptance notes

- **Task 02 (required):** every student should have a live frontend URL. Accept Vercel or Netlify; verify the env var `VITE_API_URL` is set (value may be a placeholder).
- **Tasks 03–04 (complete or document):** full marks for a live backend + cloud DB; award strong partial marks for a correct Dockerfile, CORS code, env-var plan, and a clear written description where time ran out.
- **Secrets:** deduct if real secret values appear in the document or in committed files. Reinforce that env-var screenshots must hide values.
- **Task 05:** require evidence of a green deploy status and at least one logs/monitoring screenshot — monitoring is the part students most often skip.

Lecturer Notes — How to Conduct This Lab

Before the lab

- Ask students to create `Vercel`, `Render`, and `Neon` (or Supabase) accounts **in advance** — sign-up and email verification eat lab time.
- Ensure each student has a deployable repo from earlier labs, or have them scaffold the Vite starter. Pre-check that `npm run build` works.
- Warn about free-tier limits: backend services sleep when idle (cold starts), and there are monthly hour/bandwidth caps — fine for a lab.

During the lab — what to emphasise

- **Frontend first:** get everyone a live URL early (Task 02) for a guaranteed win, then tackle the backend and database.
- **Environment variables, not code:** the production URL, connection string, and JWT key are all env vars — this is the central production-config lesson.
- **The three tiers connect by configuration:** frontend `VITE_API_URL` → backend; backend `ConnectionStrings__Default` → database.
- **Monitoring counts:** insist students open the logs and deploy-status pages — a deployment you cannot observe is not finished.
- **No secrets in submissions:** remind them repeatedly to hide values in screenshots and never commit `.env`.

Common deployment errors & how to guide

- **CORS failures:** the backend must list the exact frontend origin (https, no trailing slash) and be redeployed.
- **Frontend shows no data:** `VITE_API_URL` missing or wrong, or set after the build — Vite bakes env vars at build time, so redeploy after changing it.
- **Backend build fails:** the Dockerfile DLL name must match the project (`CampusEats.Api.dll`); the app must listen on `0.0.0.0` and the platform port.
- **Database connection fails:** managed Postgres needs `SSL Mode=Require`; migrations must be applied to the cloud DB.
- **Cold-start confusion:** reassure students the slow first request on a sleeping free-tier service is expected, not a bug.

Pacing & support

- Give a 5-minute warning before each task. If the class is behind, guarantee the frontend deployment and the monitoring review; let the backend/database be documented conceptually.
- Reserve the final 6 minutes for the quick test, screenshots (values hidden), and CourseWeb upload including the deployment URLs.

Lab Connection & References

Lab connection: this lab completes the CampusEats journey — from building the frontend (Lab 02), API (Lab 03), and database/auth (Lab 04), through version control and CI (Lab 08), to a live deployment here. Together they are the full delivery pipeline assessed in the Main Assignment.

- **Lecture 11 slide deck —** SE3090: DevOps and Cloud Hosting Platforms (primary reference).

- Vercel Docs (vercel.com/docs) • Render Docs (render.com/docs) • Neon Docs (neon.tech/docs) • Netlify / Railway / Supabase documentation.
- Microsoft Learn — 'Host and deploy ASP.NET Core' and configuration / environment variables in .NET.
- The Twelve-Factor App (12factor.net) — config in the environment • OWASP guidance on secret management.